

COMPRESS

Describe what stays and what goes

01 / THE SAS EXAMPLE

```
data test;
  infile datalines trunccover;
  input text $char20.;
  no_spaces = compress(text);
  no_digits = compress(text, '0123456789');
  only_letters = compress(text, '', 'ka');
  datalines;
ABC 123 XYZ
Phone: 555-1234
ID-2023-Q4
;
run;
proc print data=test noobs;
run;
```

REPORTED TEST SETUP / 21 SEP 2026

Claude Sonnet 4.5 / AWS Bedrock GovCloud
Claude Code 2.1.278 / effortLevel=high / prompt v0.1.0

ATTACH SOURCE OR PASTE IT AFTER THE PROMPT. START FRESH EACH TIME.

WHAT YOU WILL PRACTICE

Replace vague cleaning instructions with concrete character rules.

Three strings illustrate three rules: remove spaces, remove digits, or keep letters. For these ASCII inputs, ka keeps alphabetic characters. Specify what cleaning should preserve.

EXPECTED RESULT

Row	no_spaces	no_digits	only_letters
1	"ABC123XYZ"	"ABC XYZ"	"ABCXYZ"
2	"Phone:555-1234"	"Phone: -"	"Phone"
3	"ID-2023-Q4"	"ID- -Q"	"IDQ"

Quotes mark string boundaries and are not data. Stored trailing padding is omitted. Preserve both internal spaces in the first no_digits result.

Reference values checked in Python; SAS execution pending.

REPORTED EXPECTED-VALUE MATCHES

Prompt A 2/3 | Prompt B 3/3

Prompt A: leave choices open

High degrees of freedom / Notice the decisions the model makes for you.

COPY THIS PROMPT + THE SAS EXAMPLE

Convert this SAS code to Python.
Show code in this response. Do not create or run files, or read existing Python files.

REPORTED TRIAL RESULTS / 21 SEP 2026

One pattern kept the wrong characters

Run	only_letters pattern	Values match
A1	[^A-Za-z0-9]	No
A2	[^a-zA-Z]	Yes
A3	[^a-zA-Z]	Yes

A1: DIGITS SURVIVED IN EVERY ROW

"Phone: 555-1234" became "Phone5551234".

The required result is "Phone".

Local replay of the reported expression reproduces this error.

SOURCES: results-20260921.csv; trial-log-20260921.md; analysis-20260921.md

WHY CHOICES REMAIN OPEN

- 1

The source still needs interpretation

A1 kept letters and digits. A2 and A3 kept letters only and reportedly matched all three result columns.
- 2

Spell out the character class

For this ASCII exercise, ka keeps A-Z and a-z. Adding 0-9 to the allowed set changes the meaning.
- 3

Use inputs that expose the difference

Each row contains digits, so the letters-only check catches A1. Compare exact strings, including spaces.

ANTI-PATTERN: "CLEAN UP THE STRINGS"

Name what stays and what goes for each result. Removing digits must preserve punctuation and internal spaces.

A neat-looking result can still contain wrong characters.

Prompt B: make the choices explicit

Low degrees of freedom / State the behavior, then ask for a check.

COPY THIS PROMPT + THE SAME SAS EXAMPLE

Convert this SAS code to Python using pandas.
Show code in this response. Do not create or run files, or read existing Python files.

- Recreate the three text values in their original order.
- These inputs contain ASCII characters only. Ignore SAS trailing storage padding when comparing values; preserve internal spaces.
- no_spaces: remove literal spaces with `.str.replace(' ', '', regex=False)`.
- no_digits: remove 0-9 with `.str.replace('[0-9]', '', regex=True)`.
- only_letters: ka keeps letters; use `.str.replace('[^A-Za-z]', '', regex=True)`.
- Include assertions for these three complete result columns:
no_spaces: ['ABC123XYZ', 'Phone:555-1234', 'ID-2023-Q4']
no_digits: ['ABC XYZ', 'Phone: -', 'ID--Q']
only_letters: ['ABCXYZ', 'Phone', 'IDQ']
- Return one complete script. Compute results from text; do not hard-code the result columns.

Briefly explain the difference between removing and keeping.

WHY THESE DETAILS HELP

- 1

Separate removal from keeping
Spaces, digits and letters each get a rule. The letters-only pattern explicitly excludes digits.
- 2

Define the comparison scope
ASCII inputs only. Omit SAS trailing storage padding, while preserving internal spaces and punctuation.
- 3

Check all three result columns
B1-B3 report the requested patterns and passing assertions. Each exact string is part of the check.

REPORTED RESULTS / THREE B RUNS

Run	Values match	Assertions
B1	Yes	Passed
B2	Yes	Passed
B3	Yes	Passed

CHECK IT YOURSELF
Check "ABC XYZ": two internal spaces.
The letters-only column must contain no digits.

One behavior, three implementations

Three separate character rules remove literal spaces, remove digits, or keep ASCII letters. The translations preserve the original text and calculate each result.

SAS / SOURCE BEHAVIOR

```
%macro transform(in=, out=);
data &out;
  set &in;
  no_spaces = compress(text);
  no_digits = compress(text,
    '0123456789');
  only_letters = compress(text, '', 'ka');
run;
%mend;
```

COMPRESS ka keeps letters. Fixed-width character results retain storage padding; comparison omits trailing padding. SAS harness is unexecuted.

PYTHON / PANDAS

```
def translate(df):
    out = df.copy(deep=True)
    text = out["text"].str
    out["no_spaces"] = text.replace(
        " ", "", regex=False)
    out["no_digits"] = text.replace(
        "[0-9]", "", regex=True)
    out["only_letters"] = text.replace(
        "[^A-Za-z]", "", regex=True)
    return out
```

pandas uses literal replacement for spaces and regex character classes for digits/letters. A copy preserves the input DataFrame.

R / BASE

```
translate <- function(df) {
  out <- df
  out$no_spaces <- gsub(
    " ", "", out$text, fixed=TRUE)
  out$no_digits <- gsub(
    "[0-9]", "", out$text)
  out$only_letters <- gsub(
    "[^A-Za-z]", "", out$text)
  out
}
```

Base R gsub replaces every match. fixed=TRUE treats the space literally; the two character-class patterns use regular expressions.

Keep and remove are different rules

[^A-Za-z] removes every non-letter. Including 0-9 in the allowed class wrongly keeps digits; removing digits must retain punctuation and internal spaces.

Compare the intended characters

ASCII only. Quotes in the CSV preserve leading and internal blanks. SAS trailing storage padding is excluded; Python/R checks preserve literal strings.

SAS COMPRESS

pandas str.replace

R gsub

Three checks that earn the pass

Use the same inputs and expected behavior in each language. Check the contract as well as the values.

T1 / Original fixture: all nine strings

INPUT / CONDITION

Original three text values, including "ABC 123 XYZ" and "Phone: 555-1234".

EXPECTED

All three CSV result columns match, including "ABC XYZ" (two spaces), "Phone: -" and letters-only "Phone".

WHAT THIS CATCHES

Keeping digits for ka, dropping punctuation, or collapsing required internal spaces.

T2 / Added ASCII character boundaries

INPUT / CONDITION

Empty text; three spaces; "123"; "aB_9- !"; " A 1 B".

EXPECTED

Empty/blank rules, digit removal, punctuation retention and letters-only filtering match every expected CSV string.

WHAT THIS CATCHES

Alphanumeric filtering, only-first-match replacement, stripping leading blanks or conflating the three operations.

T3 / Preserve text, types and order

INPUT / CONDITION

All CSV strings in reverse order with custom row identifiers.

EXPECTED

Source text unchanged; result columns remain character data; row order and all expected strings retained.

WHAT THIS CATCHES

Input mutation, sorting, missing-string coercion or hard-coded result placement.

REFERENCE CODE + TESTS: `module-07-compress/appendix/`

Run: `python tools/run_appendix_tests.py --modules 07`

New reference code, separate from historical trials. SAS checks await execution; scope is stated in the companion README.